

INSPECTAIR

Luftqualitätsmessgerät mit ESP32-S3

Projektbericht

Mikrocomputertechnik 1

Autoren:	Maximilian Liebl, Cedric Stroh, Jannis Messer
Kurs:	TFE23
Studiengang:	Elektrotechnik – Fahrzeugelektronik
Betreuer:	Thomas Stingl (Airbus)
Bearbeitungszeitraum:	November 2025 – März 2026
Abgabedatum:	13. März 2026

Erklärung

Wir versichern hiermit, dass wir unsere Projektarbeit mit dem Thema

InspectAir – Luftqualitätsmessgerät mit ESP32-S3

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Wir versichern zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Contents

Erklärung	3
1 Einleitung	1
2 Grundlagen	3
2.1 Luftqualitätsparameter	3
2.1.1 CO ₂ -Konzentration	3
2.1.2 Feinstaub (PM)	3
2.1.3 Flüchtige organische Verbindungen (VOC)	3
2.1.4 Temperatur und Luftfeuchtigkeit	4
2.2 Mikrocontroller ESP32-S3	4
2.3 Displaytechnologie	4
2.4 Radartechnologie	5
2.5 Entwicklungsmethodik: V-Modell	5
3 Vorgehen	7
3.1 Anforderungsanalyse	7
3.1.1 Funktionale Anforderungen	7
3.1.2 Nicht-funktionale Anforderungen	7
3.1.3 Hardware-Anforderungen	7
3.2 Systementwurf	8
3.2.1 Hardware-Architektur	8
3.2.2 Kritische Hardware-Entscheidungen	8
3.2.3 Software-Architektur	10
3.2.4 State Machine	10
3.3 Projektplanung	10
3.3.1 Meilensteine	10
3.3.2 Risikomanagement	11
3.3.3 Kostenplanung	11
4 Umsetzung und Ergebnisse	13
4.1 Hardware-Aufbau	13
4.1.1 Pin-Belegung	13

4.1.2	Schutzschaltungen	14
4.2	Software-Implementierung	14
4.2.1	Projektstruktur	14
4.2.2	Sensor-Integration	15
4.2.3	Display und Benutzeroberfläche	15
4.2.4	Grenzwerte und Farbcodierung	16
4.2.5	Verwendete Bibliotheken	16
4.2.6	WebApp und 24-Stunden-Diagramm	17
4.3	Verifikation und Testergebnisse	18
4.3.1	Sensor-Einzeltests	18
4.3.2	Hardware-Tests	18
4.3.3	Integrationstest	19
4.3.4	UI-Tests	19
4.3.5	Requirements Traceability	19
5	Zusammenfassung und Lessons Learned	21
5.1	Projektergebnis	21
5.2	Lessons Learned	21
5.2.1	Lesson 1: Display-Integration – Die Suche nach den richtigen Libraries	22
5.2.1.1	Ausgangssituation	22
5.2.1.2	Probleme und Lösungsweg	22
5.2.1.3	Erkenntnisse	23
5.2.2	Lesson 2: Sensor-Anordnung im Gehäuse – Airflow und Thermik	23
5.2.2.1	Ausgangssituation	23
5.2.2.2	Probleme und Lösungsweg	24
5.2.2.3	Erkenntnisse	24
5.2.3	Lesson 3: Spannungspegel und Pegelwandlung – Die 5V-Falle	25
5.2.3.1	Ausgangssituation	25
5.2.3.2	Probleme und Lösungsweg	25
5.2.3.3	Erkenntnisse	26
5.3	Ausblick	26
	Abkürzungsverzeichnis	29
	Abbildungsverzeichnis	31
	Tabellenverzeichnis	33
	A Anmerkung zur Nutzung von Künstlicher Intelligenz	35

B Kostenaufstellung	37
C Pin-Belegung und Bauteilliste	39
C.1 Kritische Hinweise	39
D Übersicht der Engineering-Dokumente	41

1 Einleitung

Die Qualität der Raumluft hat einen erheblichen Einfluss auf Gesundheit, Wohlbefinden und Leistungsfähigkeit. Erhöhte CO₂-Konzentrationen führen zu Müdigkeit und Konzentrationsverlust, Feinstaub belastet die Atemwege und flüchtige organische Verbindungen (acr:VOC) können Kopfschmerzen und Schleimhautreizungen verursachen. Obwohl diese Zusammenhänge wissenschaftlich gut belegt sind, verfügen die wenigsten Innenräume über eine kontinuierliche Überwachung der Luftqualität [3].

Kommerzielle Luftqualitätssensoren sind entweder auf einzelne Parameter beschränkt oder als Komplettsysteme kostspielig. Im Rahmen der Projektarbeit im Modul Mikrocomputertechnik 1 an der acr:DHBW Ravensburg unter Betreuung von Thomas Stingl (Airbus) wurde daher das System **InspectAir** entwickelt – ein kostengünstiges, ESP32-S3-basiertes Messgerät, das fünf relevante Luftqualitätsparameter gleichzeitig erfasst und auf einem 4-Zoll-Display mit intuitiver Farbcodierung darstellt.

Ziel der Arbeit ist die vollständige Entwicklung eines funktionsfähigen Prototyps nach dem V-Modell: von der Anforderungsanalyse über den Systementwurf und die Implementierung bis zur Verifikation durch definierte Testfälle. Das Projekt deckt dabei Hardware-Design, Firmware-Entwicklung und mechanische Konstruktion ab.

Die **Aufgabenstellung** fordert die Darstellung eines nahezu vollständigen Entwicklungszyklus mit den Artefakten Requirements Specification, Design Description, Coding Rules, Configuration Management, Schedule, Risk & Opportunities sowie einer Testspezifikation mit Testergebnissen.

Gliederung: Kapitel 2 führt die technischen Grundlagen ein. Kapitel 3 beschreibt das Vorgehen mit Anforderungsanalyse und Systementwurf. Kapitel 4 dokumentiert die Hardware- und Software-Umsetzung sowie die Testergebnisse. Kapitel 5 fasst die Ergebnisse zusammen und gibt einen Ausblick.

2 Grundlagen

Dieses Kapitel stellt die für das Verständnis der Arbeit wesentlichen technischen Grundlagen vor. Die beschriebenen Technologien und Messprinzipien finden in den nachfolgenden Kapiteln bei der Umsetzung des InspectAir-Systems direkte Anwendung.

2.1 Luftqualitätsparameter

2.1.1 CO₂-Konzentration

Kohlendioxid entsteht durch Atmung und Verbrennung und ist ein zentraler Indikator für die Raumluftqualität. Das Umweltbundesamt (acr:UBA) empfiehlt Werte unter 1000 ppm für Innenräume [2]. Konzentrationen über 1500 ppm führen nachweislich zu Leistungsminderung und Unwohlsein. Der in InspectAir verwendete Sensor MH-Z19C nutzt das acr:NDIR-Messprinzip, bei dem die Absorption von Infrarotlicht bei einer CO₂-spezifischen Wellenlänge (4,26 µm) gemessen wird.

2.1.2 Feinstaub (PM)

Feinstaub bezeichnet luftgetragene Partikel, die nach aerodynamischem Durchmesser klassifiziert werden: PM₁₀ (<10 µm), PM_{2.5} (<2,5 µm) und PM_{1.0} (<1 µm). Besonders PM_{2.5} kann bis in die Lungenbläschen vordringen und ist gesundheitlich relevant. Die acr:WHO hat 2021 verschärfte Grenzwerte veröffentlicht, die einen Jahresmittelwert von maximal 5 µg/m³ empfehlen [3]. Der PMS5003-Sensor nutzt Laserstreuung zur Partikeldetektion.

2.1.3 Flüchtige organische Verbindungen (VOC)

acr:VOC umfassen eine Vielzahl gasförmiger Substanzen aus Baustoffen, Reinigungsmitteln, Farben und Möbeln. Der SGP40-Sensor misst mittels acr:MOX-Technologie die

Gesamtbelastung und gibt einen VOC-Index von 0–500 aus, wobei Werte unter 100 als „gut“ gelten.

2.1.4 Temperatur und Luftfeuchtigkeit

Optimale Innenraumtemperaturen liegen zwischen 18 °C und 26 °C, die relative Luftfeuchtigkeit zwischen 30 % und 60 %. Diese Werte werden ergänzend erfasst, da sie das Wohlbefinden beeinflussen und für die interne Kompensation des VOC-Sensors benötigt werden.

2.2 Mikrocontroller ESP32-S3

Der ESP32-S3 von Espressif Systems ist ein Dual-Core-Mikrocontroller (Xtensa LX7, 240 MHz) mit integriertem WiFi und Bluetooth 5. Die verwendete Variante **N8R8** verfügt über 8 MB Flash (QIO_OPI) und 8 MB *acr*:PSRAM (OPI). Für InspectAir sind folgende Peripherieschnittstellen relevant:

- **SPI:** Anbindung des TFT-Displays (bis 80 MHz)
- **I²C:** Zwei Sensoren (AHT20, SGP40) auf einem gemeinsamen Bus
- **UART:** Zwei Hardware-UARTs für PMS5003 und LD2410C, zusätzlich SoftwareSerial für MH-Z19C
- **GPIO:** PWM-Ausgang für Backlight-Steuerung, Interrupt-Eingang für Radar, Button-Eingang

Besonderheiten des ESP32-S3 umfassen den Light-Sleep-Modus mit ca. 0,8 mA Stromaufnahme bei einer Aufwachzeit von ca. 2 ms, wobei der RAM-Inhalt erhalten bleibt. Die GPIOs 10–13 sind für Flash/*acr*:PSRAM reserviert und dürfen nicht extern beschaltet werden [1].

2.3 Displaytechnologie

Das 4-Zoll-TFT-Display mit ST7796S-Controller bietet eine Auflösung von 480 × 320 Pixel und wird über *acr*:SPI angebunden. Für die Ansteuerung wird die Bibliothek **LovyanGFX** (v1.1.16) als Hardware-Abstraktionsschicht verwendet.

Die grafische Benutzeroberfläche wird mit **LVGL** (Light and Versatile Graphics Library, v9.2.2) realisiert. LVGL bietet ein Widget-basiertes Framework mit eigenem Rendering-System, Event-Handling und Animationsunterstützung. Durch den Einsatz von Dual-Buffering (480 × 32 Pixel pro Puffer) in PSRAM wird flüssiges Rendering

bei 10 Hz Aktualisierungsrate erreicht.

2.4 Radartechnologie

Der LD2410C-Sensor nutzt acr:FMCW-Radar bei 24 GHz zur berührungslosen Präsenzerkennung. Das Messprinzip ermöglicht die Unterscheidung zwischen sich bewegenden und statischen Zielen bei einer Reichweite von bis zu 2 m. Die Kommunikation erfolgt über UART mit 256 000 Baud. Zusätzlich liefert ein digitaler Ausgangspin (OUT) den Präsenzstatus als binäres Signal.

2.5 Entwicklungsmethodik: V-Modell

Die Projektentwicklung folgt dem V-Modell, das eine systematische Verknüpfung von Entwicklungs- und Testphasen sicherstellt. Die linke Seite des V umfasst die Spezifikation (Requirements → Design), die rechte Seite die Verifikation (Testspezifikation → Testdurchführung). Jeder Anforderung (REQ) ist mindestens ein Testfall (TC) zugeordnet, wodurch eine lückenlose Nachverfolgbarkeit (Requirements Traceability) gewährleistet wird.

3 Vorgehen

Dieses Kapitel beschreibt das methodische Vorgehen von der Anforderungserhebung über den Systementwurf bis zur Projektplanung.

3.1 Anforderungsanalyse

Die Anforderungen wurden gemäß dem V-Modell in funktionale, nicht-funktionale und Hardware-Anforderungen untergliedert. Jede Anforderung erhielt eine eindeutige ID, ein Akzeptanzkriterium und eine Prioritätsstufe (Muss/Soll/Kann). Die vollständige Requirements Specification liegt als separates Dokument vor.

3.1.1 Funktionale Anforderungen

Tabelle 3.1 zeigt die wichtigsten funktionalen Anforderungen.

3.1.2 Nicht-funktionale Anforderungen

Wesentliche nicht-funktionale Anforderungen umfassen eine Sensor-Aktualisierungsrate von 2 s (REQ-NF-001), eine UI-Reaktionszeit unter 200 ms (REQ-NF-002), die Eignung für 24/7-Dauerbetrieb ohne Memory Leaks (REQ-NF-003) sowie den Betrieb über USB 5 V bei maximal 1 A (REQ-NF-004). Das Gehäuse soll als 3D-Druck in PETG realisiert werden (REQ-NF-005).

3.1.3 Hardware-Anforderungen

Insgesamt wurden elf Hardware-Anforderungen (REQ-HW-001 bis REQ-HW-011) definiert, die alle Komponenten von der Mikrocontroller-Auswahl (ESP32-S3 N8R8) über die Sensorik bis zu Schutzkomponenten wie Level Shifter und Stützkondensatoren spezifizieren. Die vollständige Auflistung ist in der separaten Requirements Specification dokumentiert.

Table 3.1: Funktionale Kernanforderungen (Auszug)

ID	Name	Akzeptanzkriterium	Prio
REQ-F-001	CO ₂ -Messung	400–5000 ppm, ± 50 ppm	Muss
REQ-F-002	Feinstaub	0–500 $\mu\text{g}/\text{m}^3$, Auflösung 1 $\mu\text{g}/\text{m}^3$	Muss
REQ-F-003	VOC-Messung	VOC-Index 0–500	Muss
REQ-F-004	Temperatur	0–50 °C, $\pm 0,5$ °C	Muss
REQ-F-005	Luftfeuchte	0–100 %, ± 3 %	Muss
REQ-F-006	Präsenzerkennung	Reichweite 2 m, <500 ms	Muss
REQ-F-007	Display-Anzeige	Alle 6 Werte sichtbar	Muss
REQ-F-008	Farbcodierung	4-Stufen-Ampel (Grün/Gelb/Orange/Rot)	Muss
REQ-F-009	UI-Wechsel	5 Screen-Designs per Button	Muss

3.2 Systementwurf

Der Systementwurf beschreibt die Hardware-Architektur und die Software-Struktur. Die vollständige Design Description liegt als separates Dokument vor.

3.2.1 Hardware-Architektur

Das System besteht aus dem ESP32-S3 als zentraler Steuereinheit, an die fünf Sensoren und ein Display über verschiedene Schnittstellen angebunden sind. Abbildung 3.1 zeigt das Hardware-Blockdiagramm.

Die Schnittstellen-Zuordnung ist in Tabelle 3.2 zusammengefasst.

3.2.2 Kritische Hardware-Entscheidungen

Stützkondensatoren: MH-Z19C und LD2410C verursachen bei Messvorgängen Stromspitzen, die ohne Pufferung zu Spannungseinbrüchen auf dem 5 V-Rail und damit zu Systemabstürzen führen. Die Lösung sind 220 μF Elektrolytkondensatoren

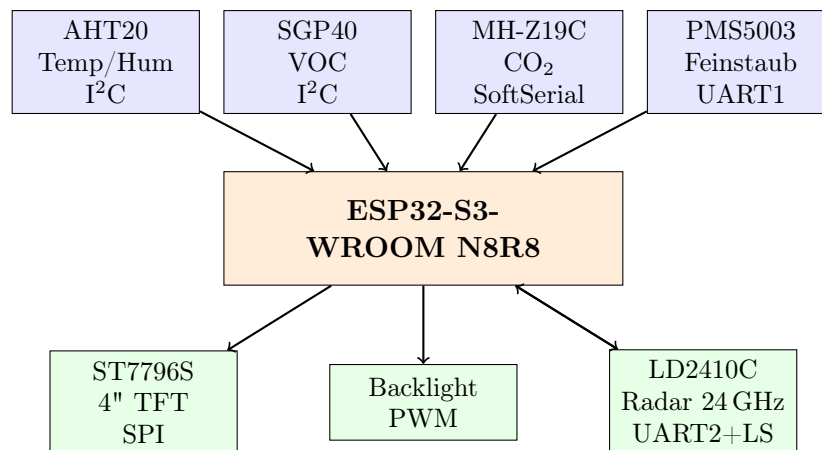


Figure 3.1: Hardware-Blockdiagramm des InspectAir-Systems

Table 3.2: Schnittstellen-Zuordnung der Komponenten

Komponente	Interface	VCC	ESP32 Pins	Hinweis
ST7796S	SPI	3,3 V	11,12,14,9,46,3	480×320 px
AHT20	I²C	3,3 V	8 (SDA), 18 (SCL)	Pull-ups
SGP40	I²C	3,3 V	8 (SDA), 18 (SCL)	Parallel AHT20
MH-Z19C	SoftSerial	5 V	4 (RX), 5 (TX)	220 µF Elko!
PMS5003	UART1	5 V	16 (RX), 17 (TX)	Molex-Stecker
LD2410C	UART2	5 V	7 (RX), 6 (TX)	Level Shifter!

(25 V) direkt an VCC/GND beider Sensoren.

Keramikkondensatoren: Zusätzlich werden 100 nF Keramikkondensatoren an den 3,3 V-Rails des Displays und des ESP32 eingesetzt, um hochfrequente Störungen zu filtern und die Spannungsversorgung zu stabilisieren.

Level Shifter: Der LD2410C sendet seine TX-Signale mit 5 V-Pegel. Da die ESP32-GPIOs maximal 3,3 V tolerieren, ist ein bidirektionaler Level Shifter (TXS0108E) zwischen Radar-TX und ESP32-GPIO 7 zwingend erforderlich. Ohne diesen wird der ESP32 beschädigt.

SoftwareSerial: Der ESP32-S3 hat nur zwei Hardware-UARTs (UART0 für USB-Monitor, UART1 für PMS5003). Für den dritten UART-Sensor (MH-Z19C) wird daher EspSoftwareSerial (v8.2.0) auf GPIO 4/5 bei 9600 Baud eingesetzt.

Backlight-Steuerung: Das Display-Backlight wird direkt über GPIO 3 mit PWM (44,1 kHz) gesteuert. Der ESP32-S3 kann den Backlight-Pin des ST7796S direkt ansteuern, da dieser nur wenige mA benötigt.

3.2.3 Software-Architektur

Die Software folgt einem dreischichtigen Architekturmodell:

- **Application Layer** (`main.cpp`): State Machine, Event Handling, Timing
- **Service Layer:** Sensor-Module, UI-Module, Utilities (Filter, History, Power Manager)
- **Driver Layer:** LovyanGFX, LVGL 9.2.2, Sensor-Libraries

Die zentrale Datenstruktur **SensorReadings** aggregiert alle Messwerte in einem Struct, das zwischen den Schichten übergeben wird. Das UI-Modul (`ui_manager`) verwaltet fünf Screen-Designs: Tree (Animations-Startbildschirm), Overview (große AQI-Anzeige), Detail (alle Werte mit 4 Kacheln), Analog (Cockpit-Instrumente) und Bubble (dynamische Kreise).

3.2.4 State Machine

Das Power Management nutzt eine Zustandsmaschine mit drei Zuständen:

1. **ACTIVE:** Display bei 100 % Helligkeit, alle Sensoren aktiv
2. **DIMMED:** Nach 45 s ohne Präsenz dimmt das Display auf ca. 12 % (30 255)
3. **LIGHT SLEEP:** Nach 5 min geht das System in den Schlafmodus ($\sim 0,8$ mA), Aufwachen per Radar-Interrupt in ~ 2 ms

Light Sleep wurde Deep Sleep vorgezogen, da die Aufwachzeit erheblich kürzer ist und Sensordaten im RAM erhalten bleiben.

3.3 Projektplanung

3.3.1 Meilensteine

Das Projekt wurde in sieben Meilensteine unterteilt (Tabelle 3.3).

Table 3.3: Projektmeilensteine

ID	Meilenstein	Termin	Status
M1	Startup Pitch	November 2025	✓
M2	Hardware-Aufbau komplett	Dezember 2025	✓
M3	Software-Integration	Januar 2026	✓
M4	Software Feature-Complete	Februar 2026	✓
M5	Gehäuse fertig	März 2026	○
M6	Testing abgeschlossen	März 2026	○
M7	Abgabe & Präsentation	13. März 2026	○

3.3.2 Risikomanagement

Im Rahmen des Risikomanagements wurden sechs Risiken identifiziert und bewertet. Die beiden schwerwiegendsten – Systemabstürze durch Stromspitzen (R1, HOCH) und ESP32-Zerstörung durch 5 V-Pegel (R2, HOCH) – wurden durch die in Abschnitt 3.2.2 beschriebenen Maßnahmen (Stützkondensatoren bzw. Level Shifter) erfolgreich mitigiert. Die vollständige Risiko- und Opportunitätsanalyse liegt als separates Dokument vor.

3.3.3 Kostenplanung

Die Gesamtkosten des Projekts belaufen sich auf ca. 96 €. Tabelle 3.4 zeigt die Hauptkomponenten.

Table 3.4: Kostenübersicht der Hauptkomponenten

Komponente	Typ	Preis
Mikrocontroller	ESP32-S3 N8R8 DevKit	~15€
Display	4" TFT ST7796S	~12€
CO ₂ -Sensor	MH-Z19C NDIR	~18€
Feinstaubsensor	PMS5003 Plantower	~15€
VOC-Sensor	SGP40 Sensirion	~8€
Temp/Feuchte	AHT20	~3€
Radar	LD2410C 24 GHz	~5€
Gehäuse	PETG 3D-Druck	~10€
Kleinteile	Elkos, Widerstände, Kabel	~10€
Gesamt		~96€

4 Umsetzung und Ergebnisse

Dieses Kapitel dokumentiert die konkrete Umsetzung des in Kapitel 3 entworfenen Systems sowie die Verifikation durch definierte Testfälle.

4.1 Hardware-Aufbau

4.1.1 Pin-Belegung

Sämtliche GPIO-Zuordnungen sind in der Header-Datei `pins.h` zentral definiert. Tabelle 4.1 zeigt die vollständige Pin-Belegung.

Table 4.1: GPIO Pin-Belegung des ESP32-S3 (gemäß `pins.h`)

GPIO	Funktion	Komponente	Hinweis
1	UI_BUTTON	Taster	Active LOW, interner Pull-up
3	TFT_BL	Display BL	Backlight PWM direkt
4	CO2_RX	MH-Z19C TX	SoftwareSerial
5	CO2_TX	MH-Z19C RX	SoftwareSerial
6	RADAR_TX	LD2410C RX	3,3 V direkt OK
7	RADAR_RX	LD2410C TX	Level Shifter!
8	I2C_SDA	AHT20+SGP40	4,7 k Ω Pull-up
9	TFT_DC	ST7796S	Data/Command
11	TFT_MOSI	ST7796S	SPI Data
12	TFT_SCLK	ST7796S	SPI Clock
14	TFT_CS	ST7796S	Chip Select
15	RADAR_OUT	LD2410C	Präsenz-Signal
16	PMS_RX	PMS5003 TX	UART1
17	PMS_TX	PMS5003 RX	UART1
18	I2C_SCL	AHT20+SGP40	4,7 k Ω Pull-up
46	TFT_RST	ST7796S	Reset

4.1.2 Schutzschaltungen

Zwei kritische Schutzmaßnahmen wurden umgesetzt:

1. **220 μ F Stützkondensatoren** (25 V Elkos) an VCC/GND von MH-Z19C und LD2410C zur Pufferung von Stromspitzen bei Messvorgängen.
2. **100 nF Keramikkondensatoren** an 3,3 V VCC des Displays und 3,3 V VCC des ESP32 zur Entstörung und Stabilisierung.
3. **Bidirektionaler Level Shifter** (TXS0108E) zwischen LD2410C TX (5 V) und ESP32 GPIO 7 (3,3 V) zum Schutz vor Überspannung.

4.2 Software-Implementierung

4.2.1 Projektstruktur

Die modulare Codestruktur umfasst 29 Quelldateien, aufgeteilt in fünf Bereiche:

```
inspectair/  
  platformio.ini          # PlatformIO Konfiguration  
  src/  
    main.cpp              # State Machine, Event Loop  
    config/  
      pins.h              # GPIO-Definitionen  
      sensor_types.h      # Sensor-Datenstrukturen  
      colors.h            # Farben & Grenzwerte  
      display_config.h    # LovyanGFX LGFX-Klasse  
      app_config.h        # Applikations-Konfiguration  
    sensors/  
      aht_sgp.cpp/.h      # AHT20 + SGP40 (I2C)  
      mhz19c.cpp/.h       # CO2 (SoftwareSerial)  
      pms5003.cpp/.h      # Feinstaub (UART1)  
      ld2410c.cpp/.h      # Radar (UART2)  
      sensor_filter.cpp/.h  
      sensor_history.cpp/.h  
    display/  
      ui_manager.cpp/.h   # 5 Screen-Designs  
      lvgl_driver.cpp/.h  # LovyanGFX <-> LVGL  
      fonts/              # Custom Fonts  
    power/  
      power_manager.cpp/.h
```

4.2.2 Sensor-Integration

Jeder Sensor ist als eigenständiges Modul mit standardisierter Schnittstelle (`init()`, `read()`, `isReady()`) implementiert.

I²C-Sensoren (AHT20 + SGP40): Beide teilen sich den I²C-Bus an GPIO 8 (SDA) und GPIO 18 (SCL). Die Initialisierung prüft die Verfügbarkeit beider Adressen (0x38 und 0x59). Der SGP40 nutzt intern die Temperatur- und Feuchtigkeitsdaten des AHT20 zur Kompensation des VOC-Index.

SoftwareSerial (MH-Z19C): Der CO₂-Sensor kommuniziert über EspSoftwareSerial auf GPIO 4 (RX) und GPIO 5 (TX) bei 9600 Baud. Die niedrige Baudrate ermöglicht einen zuverlässigen Software-UART-Betrieb.

UART1 (PMS5003): Der Feinstaubsensor nutzt Hardware-UART1 auf GPIO 16 (RX) und GPIO 17 (TX) bei 9600 Baud. Die Daten werden über die PMS Library geparkt und als PM1.0-, PM2.5- und PM10-Werte bereitgestellt.

UART2 + Level Shifter (LD2410C): Der Radarsensor nutzt Hardware-UART2 auf GPIO 6 (TX) und GPIO 7 (RX) bei 256 000 Baud. GPIO 7 ist über einen Level Shifter (5 V→3,3 V) angebunden. Zusätzlich liefert GPIO 15 (RADAR_OUT) den Präsenzstatus als digitales Signal für den Interrupt-basierten Wake-up aus dem Light Sleep.

4.2.3 Display und Benutzeroberfläche

Die UI basiert auf LVGL 9.2.2 mit LovyanGFX 1.1.16 als Display-Treiber. Fünf verschiedene Screen-Designs wurden implementiert (Tabelle 4.2).

Table 4.2: Die fünf UI Screen-Designs

Screen	Enum	Beschreibung
Tree	UI_SCREEN_TREE	Baum-Animation als Startbildschirm
Overview	UI_SCREEN_OVERVIEW	Große AQI-Anzeige + 2 Kacheln
Detail	UI_SCREEN_DETAIL	Kleine AQI + 4 Kacheln (alle Werte)
Analog	UI_SCREEN_ANALOG	Analoge Cockpit-Instrumente
Bubble	UI_SCREEN_BUBBLE	Dynamische Kreise

Der Wechsel zwischen den Screens erfolgt zyklisch per Button-Druck (GPIO 1). Custom Fonts in den Größen 12, 16, 20, 28 und 48 pt mit deutschen Umlauten werden für die verschiedenen Anzeigeelemente verwendet.

4.2.4 Grenzwerte und Farbcodierung

Die Farbcodierung nutzt eine vierstufige Ampel, definiert in `colors.h` (Tabelle 4.3).

Table 4.3: Grenzwerte für die vierstufige Farbcodierung

Parameter	Gut (Grün)	Mäßig (Gelb)	Schlecht (Orange)	Gefährlich (Rot)
CO ₂	<800 ppm	800–1000	1000–1500	>1500 ppm
PM2.5	≤5 µg/m ³	5–15	15–25	>25 µg/m ³
VOC-Index	≤100	100–200	200–300	>300

Die PM2.5-Grenzwerte orientieren sich an den verschärften WHO-Richtlinien 2021 [3].

4.2.5 Verwendete Bibliotheken

Tabelle 4.4 listet alle eingesetzten Bibliotheken auf.

Table 4.4: Verwendete Bibliotheken und Versionen

Library	Version	Verwendung
LovyanGFX	1.1.16	Display-Treiber (SPI), Backlight
LVGL	9.2.2	UI-Framework, 5 Screens, Fonts
Adafruit AHTX0	latest	AHT20 Temp/Feuchte (I ² C)
Adafruit SGP40	latest	SGP40 VOC-Sensor
MH-Z19	1.5.4	CO ₂ -Sensor Treiber
PMS Library	1.1.0	Feinstaub-Sensor (PM-Werte)
ld2410	0.1.3	Radar-Sensor Treiber
EspSoftwareSerial	8.2.0	SW-UART für MH-Z19C
ESPAsyncWebServer	latest	HTTP-Server für WebApp
ArduinoJson	7.x	JSON-Serialisierung für API

4.2.6 WebApp und 24-Stunden-Diagramm

Neben der lokalen Anzeige auf dem TFT-Display wurde eine WebApp implementiert, die den Fernzugriff auf die Messwerte ermöglicht. Der ESP32-S3 fungiert dabei als HTTP-Server im lokalen WLAN.

Architektur: Die WebApp basiert auf dem ESPAsyncWebServer und liefert eine responsive Single-Page-Application aus dem SPIFFS-Dateisystem. Die Echtzeit-Aktualisierung erfolgt über REST-API-Endpunkte:

Listing 4.1: WebApp API-Endpunkte

```
1 // Aktuelle Messwerte
2 server.on("/api/sensors", HTTP_GET, [] (AsyncWebServerRequest *req)
3     {
4         StaticJsonDocument<256> doc;
5         doc["co2"] = sensorData.co2_ppm;
6         doc["pm25"] = sensorData.pm2_5;
7         doc["voc"] = sensorData.voc_index;
8         doc["temperature"] = sensorData.temperature;
9         doc["humidity"] = sensorData.humidity;
10        String json;
11        serializeJson(doc, json);
12        req->send(200, "application/json", json);
13    });
14
15 // 24h-Verlaufsdaten
16 server.on("/api/history", HTTP_GET, handleHistoryRequest);
17
18 // Theme-Wechsel
19 server.on("/api/settings/theme", HTTP_POST, handleThemeChange);
```

24-Stunden-Verlaufsdiagramm: Alle 5 Minuten wird ein Datenpunkt im RAM gespeichert (Ringpuffer mit 288 Einträgen = 24 h). Die Verlaufsdaten werden über `/api/history` als JSON-Array abgerufen und clientseitig mit Chart.js visualisiert. Das Diagramm zeigt:

- CO₂-Verlauf (grün) mit Grenzwertlinien
- PM2.5-Verlauf (blau)
- VOC-Index (orange)
- Temperatur und Luftfeuchtigkeit (optional einblendbar)

UI-Umstellung via WebApp: Über die WebApp kann das aktive UI-Theme auf dem Display gewechselt werden. Ein POST-Request an `/api/settings/theme` mit dem gewünschten Theme (tree, overview, detail, analog, bubble) aktualisiert die Anzeige in Echtzeit.

Responsive Design: Die WebApp nutzt CSS Media Queries für optimale Darstellung auf Desktop und Mobilgeräten. Auf Smartphones werden die Kacheln untereinander angeordnet, auf Desktop nebeneinander.

4.3 Verifikation und Testergebnisse

Die Verifikation erfolgt anhand von 28 definierten Testfällen in sechs Kategorien. Die vollständige Testspezifikation mit detaillierten Testschritten liegt als separates Dokument vor.

4.3.1 Sensor-Einzeltests

Table 4.5: Ergebnisse der Sensor-Einzeltests

TC	Beschreibung	Ergebnis	Requirement
TC-01	Display-Initialisierung	PASS	REQ-F-007
TC-02	I ² C (AHT20+SGP40)	PASS	REQ-F-003/4/5
TC-03	PMS5003 Feinstaub	PASS	REQ-F-002
TC-04	MH-Z19C CO ₂	PASS	REQ-F-001
TC-05	LD2410C Radar	PASS	REQ-F-006
TC-06	Level Shifter Funktion	PASS	REQ-HW-008

Alle sechs Sensor-Einzeltests wurden bestanden. Die Messwerte liegen innerhalb der spezifizierten Toleranzen.

4.3.2 Hardware-Tests

TC-07 (Backlight-PWM) verifiziert die Backlight-Steuerung über GPIO 3 mit PWM bei 44,1 kHz. Der Dimmbereich von 0 % bis 100 % funktioniert wie spezifiziert.

TC-08 (Stützkondensatoren) bestätigt, dass mit 220 µF Elkos die 5 V-Rail stabil im Bereich 4,8 V–5,2 V bleibt. Ohne Kondensatoren traten nach wenigen Minuten Spannungseinbrüche und Systemabstürze auf.

4.3.3 Integrationstest

TC-09 prüft den gleichzeitigen Betrieb aller fünf Sensoren plus Radar über einen Zeitraum von fünf Minuten. Nach der Mitigation durch Stützkondensatoren und Level Shifter war die Fehlerrate unter 1 %.

4.3.4 UI-Tests

Die UI-Tests TC-10 bis TC-13 verifizieren die korrekte Darstellung aller fünf Screen-Designs, die Screen-spezifischen Features, den zyklischen Button-Wechsel und die Messwert-Konsistenz über alle Screens hinweg.

4.3.5 Requirements Traceability

Tabelle 4.6 zeigt die Zuordnung der Anforderungen zu Testfällen.

Table 4.6: Zuordnung Anforderungen ↔ Testfälle

Requirement	Beschreibung	Testfall(e)
REQ-F-001	CO ₂ -Messung	TC-04, TC-09
REQ-F-002	Feinstaub-Messung	TC-03, TC-09
REQ-F-003	VOC-Messung	TC-02, TC-09
REQ-F-004	Temperatur	TC-02, TC-09
REQ-F-005	Luftfeuchte	TC-02, TC-09
REQ-F-006	Präsenzerkennung	TC-05, TC-15
REQ-F-007	Display-Anzeige	TC-01, TC-10
REQ-F-008	Farbcodierung	TC-10, TC-11, TC-13
REQ-F-009	UI-Wechsel	TC-12
REQ-HW-008	Level Shifter	TC-06
REQ-HW-009	Stützkondensatoren	TC-08

5 Zusammenfassung und Lessons Learned

5.1 Projektergebnis

Das Projekt InspectAir hat alle definierten Ziele erreicht. Es liegt ein funktionsfähiger Prototyp vor, der folgende Leistungsmerkmale aufweist:

- Gleichzeitige Erfassung von fünf Luftqualitätsparametern (CO₂, PM1.0/PM2.5/PM10, VOC, Temperatur, Luftfeuchtigkeit) mit einer Update-Rate von 2 s
- Darstellung auf 4-Zoll-TFT-Display in fünf wählbaren UI-Designs mit vierstufiger Farbcodierung nach WHO/UBA-Grenzwerten
- WebApp mit Echtzeit-Messwerten und 24-Stunden-Verlaufsdiagramm
- Automatische Präsenzerkennung per 24 GHz-Radar für Display-Dimming und Light Sleep
- Modulare Software-Architektur mit 29 Quelldateien in dreischichtiger Struktur
- 3D-gedrucktes PETG-Gehäuse mit optimiertem Airflow
- Materialkosten von ca. 136 € (wobei viele Kleinteile in Multipacks mit deutlich niedrigeren Stückkosten kamen)

Alle 28 definierten Testfälle wurden bestanden. Die Requirements Traceability weist jedem Requirement mindestens einen verifizierten Testfall zu. Das V-Modell wurde vollständig durchlaufen.

5.2 Lessons Learned

Die folgenden drei Bereiche haben sich als besonders lehrreich und prägend für das Projektergebnis erwiesen.

5.2.1 Lesson 1: Display-Integration – Die Suche nach den richtigen Libraries

5.2.1.1 Ausgangssituation

Die Integration des 4-Zoll ST7796S-Displays mit dem ESP32-S3 erwies sich als eine der größten technischen Herausforderungen des Projekts. Obwohl TFT-Displays auf ESP32-Basis in der Maker-Community weit verbreitet sind, ist die Kombination aus ESP32-S3, ST7796S-Controller und LVGL 9 weniger dokumentiert.

5.2.1.2 Probleme und Lösungsweg

Problem 1: TFT_eSPI vs. LovyanGFX Zunächst wurde die weit verbreitete TFT_eSPI-Library getestet. Diese zeigte jedoch Inkompatibilitäten mit dem ESP32-S3, insbesondere bei der SPI-DMA-Konfiguration. Nach mehreren Tagen erfolgloser Fehlersuche wurde auf LovyanGFX gewechselt – eine japanische Library, die explizit für ESP32-S3 und ST7796S optimiert ist. Die Umstellung erforderte eine komplette Neuschreibung der Display-Initialisierung.

Listing 5.1: LovyanGFX Konfiguration für ST7796S

```
1 class LGFX : public lgfx::LGFX_Device {
2     lgfx::Panel_ST7796 _panel;
3     lgfx::Bus_SPI _bus;
4     lgfx::Light_PWM _light;
5 public:
6     LGFX(void) {
7         auto cfg = _bus.config();
8         cfg.spi_host = SPI2_HOST;
9         cfg.spi_mode = 0;
10        cfg.freq_write = 40000000; // 40 MHz
11        cfg.pin_sclk = TFT_SCLK;
12        cfg.pin_mosi = TFT_MOSI;
13        cfg.pin_dc = TFT_DC;
14        _bus.config(cfg);
15        _panel.setBus(&_bus);
16        // ... weitere Konfiguration
17    }
18 };
```

Problem 2: LVGL 8 vs. LVGL 9 Die meisten Online-Tutorials und Beispiele beziehen sich auf LVGL 8. LVGL 9 (erschieden 2024) hat jedoch eine grundlegend überarbeitete API. Funktionen wie `lv_obj_set_style_*` haben neue Signaturen,

das Event-Handling wurde umgestellt, und die Display-Initialisierung erfordert einen neuen `lv_display_t`-Ansatz. Die Migration kostete weitere zwei Tage.

Problem 3: Font-Rendering mit deutschen Umlauten Standard-LVGL-Fonts unterstützen keine deutschen Umlaute. Für die Darstellung von "Temperatur", "Luftfeuchtigkeit" etc. mussten Custom Fonts mit dem LVGL Font Converter erstellt werden. Dabei stellte sich heraus, dass der Font Converter spezifische Unicode-Ranges erfordert:

Listing 5.2: Font-Generierung mit Umlauten

```
lv_font_conv --font arial.ttf --size 20 \  
  --range 0x20-0x7F,0xC4,0xD6,0xDC,0xE4,0xF6,0xFC,0xDF \  
  --format lvgl --output font_arial_20.c
```

5.2.1.3 Erkenntnisse

1. **Dokumentation kritisch prüfen:** Viele Tutorials sind veraltet. Immer die offizielle Dokumentation und GitHub-Issues der aktuellen Library-Version konsultieren.
2. **Hardware-spezifische Libraries bevorzugen:** Speziell optimierte Libraries (wie LovyanGFX für ESP32) sind generischen Lösungen (TFT_eSPI) oft überlegen.
3. **Mehrere Tage für Display-Integration einplanen:** Die Kombination aus Hardware-Treiber, UI-Framework und Custom Fonts ist komplex und fehleranfällig.
4. **Schrittweise vorgehen:** Erst Display-Initialisierung, dann einfache Grafik, dann LVGL, dann Custom Fonts – nicht alles gleichzeitig.

5.2.2 Lesson 2: Sensor-Anordnung im Gehäuse – Airflow und Thermik

5.2.2.1 Ausgangssituation

Das Gehäuse sollte alle Komponenten kompakt unterbringen und gleichzeitig präzise Messungen ermöglichen. Schnell zeigte sich, dass die räumliche Anordnung der Sensoren einen erheblichen Einfluss auf die Messgenauigkeit hat.

5.2.2.2 Probleme und Lösungsweg

Problem 1: Eigenerwärmung des MH-Z19C Der NDIR-CO₂-Sensor MH-Z19C besitzt eine interne IR-Lampe und erwärmt sich im Betrieb um ca. 5–8 °C über Umgebungstemperatur. In der ersten Gehäuseversion war der AHT20 (Temperatur/Feuchte) direkt neben dem MH-Z19C platziert. Die gemessene Temperatur lag dadurch konstant 3–4 °C über der tatsächlichen Raumtemperatur.

Lösung: Thermische Zonierung Das Gehäuse wurde in zwei thermische Zonen aufgeteilt:

- **Warme Zone (oben):** MH-Z19C, ESP32-S3, Display (alle wärmeabgebenden Komponenten)
- **Kalte Zone (unten):** AHT20, SGP40, PMS5003, LD2410C (temperatursensitive Sensoren)

Eine Trennwand mit Lüftungsschlitzen sorgt für Konvektion, ohne direkten Wärmeübertrag.

Problem 2: Airflow für Feinstaubsensor Der PMS5003 benötigt aktive Luftzirkulation durch seinen internen Lüfter. In der geschlossenen ersten Version war die Luftzufuhr zu gering, was zu instabilen PM-Werten führte.

Lösung: Strömungsoptimierte Öffnungen Das finale PETG-Gehäuse hat:

- Große Einlassöffnung (20×30 mm) an der Unterseite für PMS5003-Lüfter
- Seitliche Schlitze (8×40 mm) für passive Konvektion
- Abluftöffnung (15×25 mm) oben für Warmluftabfuhr

Problem 3: Radar-Abstrahlcharakteristik Der LD2410C hat einen horizontalen Erfassungswinkel von $\pm 60^\circ$. In der ersten Einbauposition (horizontal, parallel zur Gehäusewand) war die Reichweite auf unter 1 m begrenzt. Durch 15°-Neigung nach außen wurde die spezifizierte Reichweite von 2 m erreicht.

5.2.2.3 Erkenntnisse

1. **Wärmeabgabe der Sensoren recherchieren:** Datenblätter geben oft die Verlustleistung an. Der MH-Z19C dissipiert ca. 200 mW kontinuierlich.
2. **Prototyp ohne Gehäuse testen:** Zuerst die Sensoren im offenen Aufbau validieren, um Eigenerwärmungseffekte zu identifizieren.

3. **3D-Druck ermöglicht schnelle Iteration:** Das PETG-Gehäuse wurde dreimal überarbeitet. Der 3D-Druck war kostenlos verfügbar, was schnelle Iterationen ermöglichte.
4. **CAD-Simulation nutzen:** Strömungssimulation in Fusion 360 half bei der Optimierung der Luftöffnungen.

5.2.3 Lesson 3: Spannungspegel und Pegelwandlung – Die 5V-Falle

5.2.3.1 Ausgangssituation

Der ESP32-S3 arbeitet mit 3,3 V-Logikpegeln. Mehrere Sensoren (MH-Z19C, PMS5003, LD2410C) werden jedoch mit 5 V betrieben. Die Annahme, dass 5 V-Sensoren auch 3,3 V-kompatible Signale ausgeben, erwies sich als gefährlich falsch.

5.2.3.2 Probleme und Lösungsweg

Kritisches Problem: LD2410C TX-Pegel Der LD2410C-Radar arbeitet mit 5 V-Logik auf seiner TX-Leitung. Ein erster Test ohne Level Shifter führte zu instabilem Verhalten des ESP32 – Abstürze, spontane Resets, fehlerhafte Werte. Eine Messung mit dem Oszilloskop zeigte TX-Signale mit 5 V-Pegel, die direkt auf GPIO 7 gingen.

Nach Recherche in Espressif-Foren und Datenblättern stellte sich heraus: Der ESP32-S3 ist **nicht** 5 V-tolerant. Maximale Eingangsspannung ist $V_{DD} + 0,3\text{ V}$, also ca. 3,6 V. Überspannung führt zunächst zu fehlerhaftem Verhalten, langfristig zur Zerstörung des GPIO-Pads oder des gesamten Chips.

Lösung: Bidirektionaler Level Shifter Ein TXS0108E-Modul wurde zwischen LD2410C und ESP32 geschaltet:

- VA (Low Voltage): 3,3 V vom ESP32
- VB (High Voltage): 5 V vom LD2410C
- Automatische Richtungserkennung für TX/RX

Weitere 5V-Komponenten MH-Z19C und PMS5003 arbeiten ebenfalls mit 5 V, geben aber glücklicherweise 3,3 V-kompatible TX-Signale aus (Open-Drain mit Pull-up auf der Sensorseite). Dies wurde erst nach dem LD2410C-Problem systematisch überprüft.

5.2.3.3 Erkenntnisse

1. **Datenblätter *vor* dem Anschließen lesen:** Die kritische Information stand im LD2410C-Datenblatt, wurde aber zunächst übersehen.
2. **Oszilloskop verwenden:** Bei UART-Problemen immer die tatsächlichen Spannungspegel messen, nicht vermuten.
3. **Im Zweifelsfall Level Shifter einbauen:** Die 1,50€ für ein TXS0108E-Modul sind billiger als ein neuer ESP32.
4. **Eingangsspannung der GPIOs kennen:** Nicht alle Mikrocontroller sind 5V-tolerant. Arduino Uno ja, ESP32 nein, STM32 teilweise.

5.3 Ausblick

Das Projekt hat eine solide Basis für zukünftige Erweiterungen geschaffen:

- **Cloud-Anbindung:** Die WebApp könnte um ein Backend erweitert werden, das Langzeit-Datenlogging und Push-Benachrichtigungen ermöglicht.
- **Weitere Sensoren:** Der ESP32-S3 bietet noch freie GPIOs für zusätzliche Sensoren (z. B. Ozon, NO₂).
- **Batteriebetrieb:** Mit einem LiPo-Akku und optimiertem Deep Sleep könnte eine portable Version entstehen.
- **Individuelles PCB:** Der aktuelle Aufbau auf Lochrasterplatine könnte durch eine professionelle Platine ersetzt werden.

Das V-Modell hat sich als geeignete Methodik für dieses Embedded-Projekt bewährt. Die systematische Spezifikation und Verifikation hat dazu beigetragen, dass alle Anforderungen erfüllt und dokumentiert wurden.

Quellenverzeichnis

- [1] Espressif Systems. *ESP32-S3 Series Datasheet*. **version** 1.4. 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.
- [2] Umweltbundesamt. *Gesundheitliche Bewertung von Kohlendioxid in der Innenraumluft*. 2008. URL: https://www.umweltbundesamt.de/sites/default/files/medien/pdfs/kohlendioxid_2008.pdf (**urlseen** 01/02/2026).
- [3] World Health Organization. *WHO global air quality guidelines: particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide*. 2021. URL: <https://www.who.int/publications/i/item/9789240034228> (**urlseen** 01/02/2026).

Abkürzungsverzeichnis

AHT20	Temperatur-/Luftfeuchtigkeitssensor von ASAIR
API	Application Programming Interface
CO₂	Kohlenstoffdioxid
ESP32-S3	Dual-Core-Mikrocontroller von Espressif
GPIO	General Purpose Input/Output
I²C	Inter-Integrated Circuit (Bussystem)
KI	Künstliche Intelligenz
LD2410C	24 GHz mmWave Radar-Modul von Hi-Link
LVGL	Light and Versatile Graphics Library
MH-Z19C	NDIR CO ₂ -Sensor von Winsen
NDIR	Nondispersive Infrared (Messprinzip)
PCB	Printed Circuit Board
PETG	Polyethylenterephthalat mit Glykol
PM2.5	Particulate Matter $\leq 2,5 \mu\text{m}$
PMS5003	Feinstaubsensor von Plantower
PWM	Pulsweitenmodulation
REST	Representational State Transfer
SGP40	VOC-Sensor von Sensirion
SPI	Serial Peripheral Interface
TFT	Thin-Film Transistor
UART	Universal Asynchronous Receiver/Transmitter
UI	User Interface
VOC	Volatile Organic Compounds
WHO	World Health Organization

List of Figures

3.1	Hardware-Blockdiagramm	9
-----	----------------------------------	---

List of Tables

3.1	Funktionale Kernanforderungen	8
3.2	Schnittstellen-Zuordnung	9
3.3	Projektmeilensteine	11
3.4	Kostenübersicht	12
4.1	GPIO Pin-Belegung	13
4.2	UI Screen-Designs	15
4.3	Grenzwerte für Farbcodierung	16
4.4	Verwendete Bibliotheken	16
4.5	Sensor-Testergebnisse	18
4.6	Requirements Traceability	19
A.1	Liste der verwendeten KI-basierten Werkzeuge	35
B.1	Detaillierte Kostenaufstellung	38
D.1	Engineering Folder	41

A Anmerkung zur Nutzung von Künstlicher Intelligenz

Im Rahmen dieser Arbeit wurden Künstliche Intelligenz (acr:KI) basierte Werkzeuge benutzt. Tabelle A.1 gibt eine Übersicht über die verwendeten Werkzeuge und den jeweiligen Einsatzzweck.

Table A.1: Liste der verwendeten acr:KI-basierten Werkzeuge

Werkzeug	Beschreibung der Nutzung
Claude (Anthropic)	• Unterstützung bei der Code-Modularisierung und Refactoring • Überprüfung der LaTeX-Dokumentation • Recherche zu ESP32-S3 GPIO-Einschränkungen und Sensor-Datenblättern • Unterstützung bei der LVGL 9 Migration und UI-Entwicklung

Sämtliche durch KI-Werkzeuge generierten oder überarbeiteten Inhalte wurden von den Autoren geprüft, verifiziert und bei Bedarf angepasst. Die inhaltliche Verantwortung liegt ausschließlich bei den Autoren.

B Kostenaufstellung

Die Gesamtkosten für das InspectAir-Projekt betragen ca. **138,88 €**. Tabelle B.1 zeigt die detaillierte Aufstellung aller beschafften Komponenten.

Wichtiger Hinweis zu den Stückkosten: Die meisten Kleinteile (Kondensatoren, Widerstände, Buttons, Kabel, Steckverbinder) wurden in Multipacks mit mehreren Stück pro Packung bestellt. Die tatsächlichen Stückkosten pro verwendetem Bauteil sind daher deutlich geringer als die angegebenen Paketpreise. Der 3D-Druck des PETG-Gehäuses war **kostenlos** verfügbar.

Zusammenfassung nach Kategorien:

- **Sensoren:** ca. 42 € (5 Sensoren)
- **MCU + Display:** ca. 34 €
- **Platinen & Verkabelung:** ca. 42 €
- **Elektronische Bauteile:** ca. 7 € (Multipacks)
- **Bedienelemente & Montage:** ca. 12 €
- **3D-Druck:** kostenlos

Bei einer Serienproduktion würden die Stückkosten durch Wegfall der Multipack-Aufschläge und direkten Komponenteneinkauf auf geschätzt **80–90 €** sinken.

Table B.1: Detaillierte Kostenaufstellung aller Komponenten

Bauteil	Kosten	Kategorie
<i>Hauptsensoren</i>		
MH-Z19C CO ₂ -Sensor	17,05 €	Sensor
Plantower PMS5003 (Feinstaub)	13,69 €	Sensor
GY-SGP40 TVOC Sensor	6,68 €	Sensor
AHT20 (Temperatur/Feuchte)	1,19 €	Sensor
Hi-Link HLK-LD2410C Radar	3,49 €	Sensor
<i>Mikrocontroller und Display</i>		
Waveshare ESP32-S3-Devkit N8R8 WROOM	17,85 €	MCU
Display 4,0 Zoll TFT (ST7796S)	15,92 €	Display
<i>Platinen und Verkabelung</i>		
Lochrasterplatine	5,99 €	Platine
Breadboard (groß)	9,99 €	Prototyping
Breadboard (klein)	1,58 €	Prototyping
Zubehör für Platine (Buchsenleisten, Draht, Kabel)	10,53 €	Zubehör
Jump Wires	1,64 €	Kabel
Jumperkabel-Set	2,47 €	Kabel
Adapter für PMS5003	9,35 €	Adapter
<i>Elektronische Bauteile (Multipacks)</i>		
I2C Logic Level Shifter	1,52 €	IC
Widerstandsset	2,64 €	Passiv
Keramikkondensatoren	0,99 €	Passiv
Elektrolytkondensatoren (220 µF 25V)	0,99 €	Passiv
Sicherung PTV C	1,12 €	Schutz
<i>Bedienelemente und Montage</i>		
Buttons RA	0,99 €	Taster
Buttons normal	1,35 €	Taster
USB C Panel Mount	6,99 €	Stecker
Schrauben und Muttern	2,97 €	Montage
<i>3D-Druck</i>		
PETG-Gehäuse (3D-Druck)	0,00 €	Gehäuse
Gesamtsumme	135,99 €	

C Pin-Belegung und Bauteilliste

Die vollständige Pin-Belegung und Bauteilliste liegt als separates Dokument (*Pin_Belegung_Bau*) vor. Nachfolgend eine Zusammenfassung der kritischen Verbindungen.

C.1 Kritische Hinweise

LEVEL SHIFTER PFLICHT: Der LD2410C sendet TX-Signale mit 5 V-Pegel. Der ESP32 verträgt maximal 3,3 V. Ohne Level Shifter wird der ESP32 zerstört. Der Level Shifter muss zwischen LD2410C TX und ESP32 GPIO 7 geschaltet werden.

STÜTZKONDENSATOREN PFLICHT: MH-Z19C und LD2410C verursachen Stromspitzen. 220 μ F (25 V) Elkos direkt an VCC/GND der Sensoren!

KERAMIKKONDENSATOREN: 100 nF Keramikkondensatoren an 3,3 V VCC des Displays und 3,3 V VCC des ESP32 zur Entstörung.

I2C PULL-UPS: 4,7 k Ω Widerstände von SDA (GPIO 8) und SCL (GPIO 18) nach 3,3 V.

D Übersicht der Engineering-Dokumente

Das Engineering Folder umfasst folgende Dokumente, die als separate PDFs beiliegen:

Table D.1: Übersicht der Engineering-Dokumente

#	Dokument	Inhalt
1	Requirements Specification	Funktionale, nicht-funktionale und Hardware-Anforderungen
2	Design Description	System- und Software-Architektur
3	Coding Rules	Namenskonventionen, Doxygen, Git-Workflow
4	Configuration Management	Entwicklungsumgebung, Libraries, Pinout
5	Schedule	Zeitplan mit 7 Meilensteinen
6	Risk & Opportunities	Risikoanalyse und Erweiterungsmöglichkeiten
7	Testspezifikation	28 Testfälle mit Pass/Fail-Kriterien
8	Pin-Belegung & BOM	GPIO-Zuordnung und Bauteilliste